

Portal PV-ZTG - Sistema de Gestión de Flota

Conversión a React + Vite + SWC + Tailwind del prototipo HTML para la gestión de flota, auditoría Edenred, accesos, dashboard y reportes.

Stack

- Vite 8 + [@vitejs/plugin-react-swc](#)
- React 19, React Router 7
- Tailwind CSS 3
- JavaScript + JSX puro (sin TypeScript)

Criterios de interfaz

La interfaz conserva el verde institucional CFE y prioriza lectura clara, navegación predecible y controles distinguibles para sesiones operativas prolongadas. Las transiciones son breves y discretas; no se usan fondos animados ni dependencias de animación. El acceso utiliza una imagen institucional fija y un único recurso fotográfico; los indicadores de carga conservan la identidad PV-ZTG, no simulan porcentajes de avance y respetan las preferencias de movimiento reducido. En pantallas pequeñas, el contenido conserva márgenes legibles y la navegación pasa a un menú lateral.

Los títulos principales, secundarios y encabezados de tabla siguen una jerarquía tipográfica compartida. En el detalle de transacciones Edenred, la tabla conserva sus filtros visibles y desplaza las filas en un área propia con encabezado fijo.

Cómo correrlo

Frontend:

```
npm install
cp .env.example .env
```

```
npm run dev
```

Backend API:

```
cd backend  
cp .env.example .env  
npm install  
npm run dev
```

Para un arranque controlado de producción, aplica primero las migraciones versionadas y después inicia el servidor:

```
cd backend  
npm run db:migrate  
npm start
```

El backend conserva el identificador `x-request-id` en cada respuesta y registra solicitudes con error en formato JSON. No registra contraseñas, hashes ni tokens. La interfaz incluye un Error Boundary: si un componente falla, muestra una pantalla recuperable con un identificador en vez de dejar el navegador en blanco.

Backups y restauración segura

El backup se genera con las tablas y sus datos en un archivo SQL. Debe guardarse fuera del repositorio y probarse siempre en una base temporal:

```
cd backend  
$env:BACKUP_DIR="C:\pvztg-backups"  
npm run db:backup  
$env:BACKUP_FILE="C:\pvztg-backups\pvztg-YYYY-MM-DDTHH-MM-SS-sssZ.sql"  
npm run db:validate-backup  
$env:RESTORE_TARGET_DB="pvztg_restore_test"  
npm run db:restore
```

La validación comprueba que el archivo exista, no esté vacío, contenga las marcas estructurales esperadas y no incluya instrucciones `CREATE DATABASE` o `USE` que pudieran dirigir la restauración a otra base. Los respaldos se conservan fuera del repositorio; la carpeta predeterminada `backend/backups/` está excluida de Git. La

restauración ejecuta las sentencias del dump de forma individual para evitar enviar el archivo completo como un único paquete a MySQL.

Para una operación pequeña se recomienda conservar, como mínimo, respaldos diarios durante 30 días y uno mensual durante 12 meses, en una ubicación con acceso restringido. La política definitiva debe aprobarla el responsable de la dependencia según sus requisitos de retención.

La restauración contra `DB_NAME` está bloqueada por defecto. Para una restauración operativa explícita se requiere `RESTORE_ALLOW_PRODUCTION=true`, una ventana autorizada y una verificación previa del archivo.

El build de producción se comprueba con `npm run build` y se puede servir localmente con `npm run preview -- --host localhost`. No se deben usar `npm run dev` ni `SEED_RESET_PASSWORDS=true` como procesos de producción.

Los listados API aplican paginación defensiva mediante `page` y `limit`, con un máximo de 200 elementos por respuesta. Los encabezados `X-Pagination-Page`, `X-Pagination-Limit`, `X-Pagination-Total` y `X-Pagination-Total-Pages` exponen la información necesaria para que una interfaz pueda paginar sin descargar colecciones ilimitadas.

El backend valida la conexión a MySQL antes de comenzar a escuchar solicitudes. Si MySQL no está disponible o la configuración es incorrecta, el proceso termina con un mensaje explícito y no publica un servidor parcialmente funcional.

`GET /api/health` verifica que el proceso esté vivo. `GET /api/health/ready` verifica además que MySQL responda y devuelve `503` si la instancia no está lista para recibir operaciones.

Al recibir `SIGINT` o `SIGTERM`, el backend deja de aceptar conexiones nuevas, espera el cierre del servidor HTTP y libera el pool de MySQL. Esto permite reinicios controlados mediante un servicio de Windows, PM2, NSSM o el orquestador que se autorice para producción.

Reversa de una versión

Antes de desplegar una versión:

1. Ejecuta `npm run db:backup` y valida el archivo con `npm run db:validate-backup`.

2. Registra la versión del código y la última migración aplicada.
3. Detén el proceso actual de forma controlada.
4. Publica la versión anterior o la nueva según el resultado.
5. Ejecuta `GET /api/health` y `GET /api/health/ready`.
6. Sólo si existe pérdida o corrupción confirmada, restaura el backup en una base temporal para investigar. La restauración operativa requiere autorización explícita y `RESTORE_ALLOW_PRODUCTION=true`.

No se debe revertir código y esquema de base de datos de forma independiente si la versión anterior no es compatible con las migraciones nuevas. Toda migración destructiva futura debe incluir antes un plan de reversa aprobado.

Las instalaciones nuevas y las bases existentes se preparan mediante las migraciones versionadas. `schema.sql` queda como referencia/bootstrap y no se usa para actualizar una base operativa:

```
cd backend
npm run db:migrate
npm run db:seed
```

No se debe ejecutar el backend esperando que cree o modifique tablas durante el arranque.

La migración `008-retire-password-recovery.sql` elimina la tabla de tokens temporales del flujo de recuperación retirado. Antes de promoverla a una base operativa, genera y valida el backup siguiendo el procedimiento anterior.

La migración `010-ecosystem-persistence.sql` guarda documentos de las unidades en MySQL, añade el orden de aplicación de cargas Edenred y agrega un índice para las consultas de flota. La alta de departamentos y la importación masiva de unidades utilizan endpoints persistentes; la importación aplica altas, cambios y su clave de idempotencia en una sola transacción. Las listas paginadas deben consumirse completas usando los encabezados `X-Pagination-*`. La relación puede cargarse de nuevo después de reiniciar el parque; las altas requieren número de serie, se validan antes de escribirse y la interfaz conserva los identificadores persistentes devueltos por la API. Las filas nuevas sin número de serie se muestran como omitidas; no se generan unidades de ejemplo. El número económico es la identidad principal de la unidad; el VIN es identificador secundario y las placas compartidas se reportan como duplicidad, pero nunca se usan para fusionar ni descartar unidades, pues distintos vehículos pueden compartirlas en la relación oficial. Los campos obligatorios del

expediente (estado, kilometraje, combustible, departamento, número económico, resguardante, marca, submarca, tipo, modelo, placas, placas vigentes del año actual, VIN, R.P.E., centro gestor, centro de costos, ubicación técnica y arrendadora) generan casos de dato faltante; el segundo resguardante es opcional. La carga acepta el encabezado de placas del año vigente y encabezados genéricos de placas vigentes. Un encabezado histórico como **PLACAS 2025** no se considera vigente en otro año. La migración **011-current-year-plates.sql** registra el año asociado al dato. La presencia del dato se determina por el texto capturado (que es libre); el metadato de año solo limita qué valores participan en la detección de duplicados. El campo de placas vigentes conserva texto libre; para detectar duplicados se extrae la serie alfanumérica de placa, ignorando separadores habituales como guiones, espacios y puntos y descartando textos de vencimiento como **VENCE EN 2029**. Se permiten hasta dos unidades con la misma serie vigente; se reporta duplicidad a partir de la tercera. Los demás duplicados se detectan en número económico, placas y VIN. La migración **013-expand-current-year-plate-text.sql** amplía la capacidad en la base para evitar que descripciones largas como **VENCE EN 2029 - HDC824H** fallen al guardarse. La migración **014-backfill-current-year-plates.sql** asocia los valores no vacíos existentes al año actual cuando aún no tenían año registrado; el texto, incluso **VENCE 2032**, no se usa para determinar el año. Los casos desaparecen automáticamente al corregir los datos y reconciliar el parque. Edenred se asocia primero por número económico; una placa compartida no se utiliza para asignar una transacción a una unidad arbitrariamente. Sólo se omiten copias con todos sus datos importables idénticos; los números económicos repetidos se muestran para revisión.

Los contactos adicionales del perfil se guardan junto con nombre, correo y celular mediante **PATCH /usuarios/perfil**; el endpoint valida cada contacto y la lectura del perfil los recupera desde MySQL. La migración **012-profile-additional-contacts.sql** agrega el campo JSON persistente. Tras actualizar el código, aplica **npm run db:migrate** desde **backend** antes de usar esta función en una base existente. Los inicios de sesión no crean sesiones locales de prueba cuando falla el backend: se requiere autenticación real para acceder a funciones conectadas a datos.

Pruebas backend

Las pruebas unitarias no requieren MySQL y no modifican datos:

```
cd backend
```

Las pruebas de integración crean y eliminan exclusivamente la base indicada en **TEST_DB_NAME** y la preparan aplicando las mismas migraciones versionadas que usa una instalación real. Por seguridad, el comando se detiene si coincide con **DB_NAME**; no deben ejecutarse contra la base operativa:

```
$env:TEST_DB_NAME="pvztg_test"
npm run test:integration
```

No existe recuperación de contraseña por correo o SMS. Si se pierde el acceso, un administrador autorizado debe restablecer la credencial mediante el procedimiento institucional fuera de la aplicación. El cambio de contraseña dentro de una sesión autenticada y el cambio obligatorio de clave inicial sí permanecen disponibles.

Las cargas Edenred se guardan dentro de una transacción MySQL: la carga y las actualizaciones de historial de las unidades se confirman juntas o se revierten juntas si alguna unidad no existe o falla.

El seed no sobrescribe contraseñas existentes por defecto. Para una recuperación explícita de las tres cuentas canónicas de desarrollo se debe ejecutar con **SEED_RESET_PASSWORDS=true**; después del acceso inicial cada credencial debe cambiar su contraseña.

Estado de las fases

El estado se separa entre implementación técnica, evidencia ejecutada y aceptación operativa. Pasar pruebas automáticas no sustituye la aprobación del procedimiento de operación.

Fase 3 — Implementación técnica completada; cierre operativo pendiente

Completado:

- Migraciones numeradas como autoridad de evolución del esquema.
- Health check vivo (</api/health>) y readiness con MySQL (</api/health/ready>).

- Backup, validación y restauración protegida por destino.
- Cierre controlado del servidor y liberación del pool de MySQL.

Pendiente de evidencia operativa:

- Elegir el administrador de procesos de Windows.
- Probar inicio, detención y reinicio con el mecanismo elegido.
- Restaurar un backup representativo en una base temporal.
- Validar migraciones sobre una copia fiel de **pvztg**.
- Definir RTO, RPO, responsables y autorización de restauración.
- Ejecutar y evidenciar la reversa de una versión.
- Probar recuperación después de detener y volver a iniciar MySQL.
- Obtener aceptación operativa firmada.

Fase 4 — Implementación validada en base temporal; medición oficial pendiente

La suite ejecutada el 25 de septiembre de 2026 contra **pvztg_test** obtuvo:

```
10 pruebas de integración
10 exitosas
0 fallos
0 omitidas
```

También se mantienen aprobadas las pruebas unitarias y estructurales del backend.
La evidencia pendiente para cerrar formalmente la fase es:

- **EXPLAIN** de las consultas principales sobre datos representativos.
- Mediciones p50/p95 de latencia y tamaño de respuestas.
- Confirmación de índices en la copia fiel de **pvztg**.
- Validación funcional de paginación con más de una página.
- Promoción controlada de migraciones a **pvztg**.

Fase 5 — Validación funcional iniciada

Ya están cubiertos automáticamente los flujos de persistencia, autorización departamental, transacciones Edenred y los tres endpoints de autenticación. La validación manual de UX por rol continúa pendiente:

- APV.
- STT.
- Jefatura.
- Estados vacíos, carga, red, sesión expirada y permisos.
- Responsive, navegación móvil y accesibilidad básica.

Fase 6 — Correcciones de seguridad implementadas; validación operativa pendiente

Se corrigieron los hallazgos de la auditoría de seguridad:

- Los comprobantes sólo aceptan rutas internas de archivos con nombre UUID y extensión permitida; se rechazan URLs `javascript:`, `data:`, externas y otros esquemas controlados por el cliente.
- Los JWT incluyen una versión de sesión. Cada solicitud protegida valida en MySQL que la cuenta siga activa y conserve su rol, departamento y versión. Cambiar contraseña, desactivar una cuenta o modificar su rol incrementa la versión e invalida tokens anteriores.
- Login de jefatura también tiene rate limiting.
- Las respuestas de autenticación no confirman la existencia de cuentas privilegiadas.
- La migración `003-session-version.sql` agrega el control de revocación a bases existentes.

Evidencia ejecutada en `pvztg_test`:

```
11 pruebas de integración
11 exitosas
0 fallos
```

La validación operativa pendiente incluye revisar secretos y permisos del servidor Windows, confirmar que `CLIENT_URL` sea la única interfaz aprobada y realizar una prueba controlada de revocación después de desactivar o cambiar el rol de un usuario.

Fase 7 — Observabilidad técnica inicial implementada

El backend conserva un `x-request-id` por solicitud y ahora registra métricas en memoria por ruta:

- total de solicitudes;
- total de respuestas con error (`4xx` y `5xx`);
- distribución de estados HTTP;
- latencia p50 y p95 global y por ruta;
- ventana de hasta 1,000 duraciones recientes por ruta.

El endpoint `GET /api/metrics` está protegido por el header `x-metrics-token`, cuyo valor debe configurarse mediante `METRICS_TOKEN`. Si el token no está configurado o no coincide, responde `404` y no revela la existencia del endpoint. Las métricas no contienen cuerpos, tokens, contraseñas ni parámetros de solicitudes.

Ejemplo de consulta local:

```
$env:METRICS_TOKEN="token-operativo-largo"
Invoke-RestMethod "http://localhost:4000/api/metrics" -Headers @{ "x-metrics-token"
= $env:METRICS_TOKEN }
```

Las métricas se reinician al reiniciar el proceso. Para producción aún falta conectarlas a una retención externa, configurar rotación de logs y definir alertas para errores, latencia y pérdida de readiness.

También está disponible `GET /api/version` para confirmar qué servicio y versión de build atienden una instancia. Sólo devuelve metadatos no sensibles: servicio, versión, entorno y hora de arranque. La versión puede fijarse con `APP_VERSION` durante el despliegue.

Fase 8 — Preparación técnica de lanzamiento

La aplicación incluye una comprobación reproducible de configuración de lanzamiento:

```
cd backend
npm run release:check
```

La comprobación falla si no se cumple cualquiera de estas condiciones:

- `NODE_ENV=production`.

- `APP_VERSION` definido.
- `JWT_SECRET` de al menos 32 caracteres.
- `CLIENT_URL` configurado como un origen HTTP/HTTPS válido, sin rutas.
- usuario de base de datos dedicado, distinto de `root`.
- contraseña de base de datos no vacía.
- `METRICS_TOKEN` de al menos 32 caracteres.

Checklist técnico de salida:

Control	Estado verificable
Integración backend contra base temporal	Pendiente en la verificación actual; requiere <code>TEST_DB_NAME</code> aislada
Suite backend sin base	43 aprobadas; 15 de integración omitidas por falta de base temporal
Build frontend	Exitoso en la verificación actual
Lint frontend	Sin errores; 2 advertencias React existentes
Migraciones versionadas	Migración 012 agregada al código; su aplicación en base de datos no está verificada
Backup validado antes de la promoción	No verificado para los cambios actuales
Health y readiness	Cubiertos por pruebas automatizadas; no se verificó el servicio de producción en esta sesión
Controles de seguridad	Cubiertos por pruebas automatizadas; no equivale a auditoría completa del despliegue
Métricas y request ID	Cubiertos por pruebas automatizadas
Configuración de producción	Requiere ejecutar <code>release:check</code> con secretos reales
Administrador de procesos Windows	Pendiente
Recuperación de contraseña por correo/SMS	Retirada; cambio autenticado de contraseña disponible
Alertas y retención externa	Pendiente
Aceptación operativa	Pendiente

Procedimiento de lanzamiento:

1. Ejecutar `npm run test:integration` con `TEST_DB_NAME` distinto de `DB_NAME`.
2. Ejecutar `npm test`, `npm run build` y `npm run lint`.
3. Ejecutar `npm run release:check` con la configuración real, sin mostrar secretos en consola.
4. Crear y validar backup de `pvztg`.
5. Aplicar sólo migraciones aprobadas y registrar su evidencia.
6. Publicar el build y reiniciar mediante el mecanismo Windows aprobado.
7. Verificar `/api/health`, `/api/health/ready` y `/api/version`.
8. Consultar `/api/metrics` con el token operativo.
9. Ejecutar smoke tests por rol.
10. Si falla un criterio, detener el lanzamiento y aplicar el procedimiento de reversa documentado.

La Fase 8 no se declara cerrada mientras falten los controles marcados como pendientes operativos. No se inventan métricas, aceptación ni pruebas de servicios externos.

Procedimiento de promoción de una migración

Una migración aprobada se prueba primero en `pvztg_test`. Sólo después se promueve a `pvztg` con un backup verificable:

```
cd backend

# 1. Prueba aislada: no debe ser igual a DB_NAME.
$env:TEST_DB_NAME="pvztg_test"
npm run test:integration

# 2. Promoción: limpiar la variable de prueba y respaldar la base oficial.
Remove-Item Env:TEST_DB_NAME -ErrorAction SilentlyContinue
$env:BACKUP_DIR="C:\pvztg-backups"
npm run db:backup

# 3. Validar el archivo generado y conservar su ruta.
$env:BACKUP_FILE="C:\pvztg-backups\pvztg-.sql"
npm run db:validate-backup

# 4. Aplicar exclusivamente migraciones versionadas a DB_NAME=pvztg.
npm run db:migrate
```

No se copian datos de `pvztg_test` a `pvztg`. La promoción aplica código y migraciones aprobadas, no datos de prueba. Si el backup no valida, la promoción se detiene.

Checklist operativo de Fase 3

Evidencia	Estado
Mecanismo de procesos Windows elegido	Pendiente
Inicio/detención/reinicio reales	Bloqueado por la decisión anterior
Restauración temporal de backup	Pendiente
Migración sobre copia fiel de <code>pvztg</code>	Pendiente
RTO/RPO y responsables aprobados	Pendiente
Reversa documentada y probada	Pendiente
Recuperación después de caída de MySQL	Pendiente
Aceptación operativa	Pendiente

Mientras exista un elemento pendiente en esta tabla, la Fase 3 conserva el estado de implementación técnica completada, pero no se declara cerrada.

Qué se portó (contenido real, no placeholders)

Las 7 páginas tienen ya el contenido completo del prototipo original, con la interactividad que antes vivía en `<script>` sueltos convertida a hooks de React:

Página	Interactividad portada
Login	Spinner de envío → <code>useState(isLoading)</code> + redirección a <code>/dashboard</code>
Auditoría Edenred	Modal de auditoría (abrir/cerrar/backdrop) → <code>useState(isAuditModalOpen)</code>
Módulo de Flota	Panel "Expediente" (slide-over) → <code>useState(selectedUnit)</code>

Página	Interactividad portada
Gestión de Accesos	Toggles de estado de usuario → <code>defaultChecked</code> (no controlados aún)
Captura Móvil	Slider "swipe to send" → componente <code>SwipeToSend</code> con Pointer Events
Dashboard	Sin JS propio en el original - solo maquetación
Reportes	Filtros (selects/fecha) → <code>useState(isRefreshing)</code> simulando recarga

Componentes compartidos

- `components/layout/SideNavBar.jsx` - unifica las 5 variantes casi idénticas de sidebar que traía el prototipo, con estado activo por ruta (`NavLink`).
- `components/layout/TopNavBar.jsx` - header superior con tabs Resumen/Operaciones/Alertas, parametrizado por página.
- `components/layout/AppLayout.jsx` - envuelve todas las páginas del panel excepto Login y Captura Móvil (pantallas standalone, sin nav global, igual que en el prototipo).
- `components/ui/SwipeToSend.jsx` - el slider de Captura Móvil.
- `components/ui/BentoGrid.jsx`, `StatusBadge.jsx` - piezas base del design system, ya preparadas para las páginas que aún no las usan.

Decisiones pendientes de confirmar

1. **Color `primary`**: el HTML original y el documento de diseño formal ("Institutional Technical Core") no coinciden - ver el comentario al inicio de `tailwind.config.js`. Se usó el documento de diseño como fuente de verdad.
2. **Modo oscuro de Accesos**: el prototipo tenía esta página en `dark` forzado (`html class="dark"`, fondo `#121212`). Se portó en modo claro para mantener consistencia visual con el resto del panel - si se quiere ese modo oscuro específico, es un ajuste de clases, no estructural.
3. **Checkboxes de Accesos**: están como `defaultChecked` (no controlados). Falta decidir si el toggle debe pegarle a una API al cambiar o solo quedar en estado local.

Pendiente real (no incluido)

- Conectar todo a datos reales - hoy todo el contenido (tablas, KPIs, logs) es el mismo mock que traía el prototipo.
- Formularios: ningún `<input>` está validado ni conectado a estado más allá de lo mencionado arriba.
- Responsive: el prototipo original ya traía clases `md:/lg:` en varios puntos y se conservaron, pero no se re-testeó pixel a pixel en mobile.